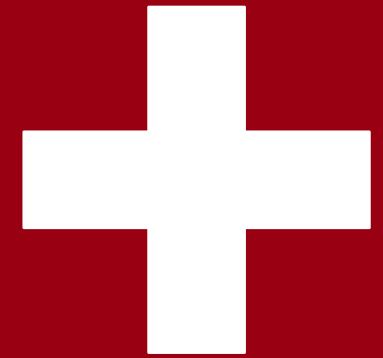


Swiss Rent Radar



Drivers of Swiss Rental Prices — Combining Official BFS Statistics with Live Listings

Kermo Aladin
Vukcevic Marko

Agenda

1

Motivation & research question

Why rent in Switzerland matters and what we aim to answer

2

Data sources

Official BFS reference + live Homegate listings

3

Methods & pipeline

Scraping, regex, pandas, SQLite, OOP

4

Statistical results

Pearson, t-test, ANOVA — all with p-values

5

Visualisations & web app

matplotlib, folium choropleth, Streamlit

6

LLM market commentary

Together.ai Mistral 7B over our data

7

Conclusions & limitations

What we found, what we'd do next

Why rent? Why now?

27.1

CHF / m²

most expensive: Genève

11.6

CHF / m²

cheapest: Jura

2.3×

spread

between richest and poorest

Research question

Which structural and geographic factors most strongly explain rent differences in Switzerland — and where do live Homegate asking prices systematically diverge from the BFS reference?

Two complementary data sources

REST API

opendata.swiss / BFS

Federal Statistical Office

Mean rent per canton (4-room flats)

Mean price per m²

Cantonal population (weighting)

26 cantons · annual reference figures

WEB SCRAPER

Homegate.ch

Live apartment listings

Parsed from __NEXT_DATA__ JSON

BeautifulSoup + lxml

Polite rate-limit (1 req/sec)

Snapshot fallback for reproducibility

Combining both lets us measure where current asking prices deviate from the official mean.

End-to-end pipeline



Regex parsing (bonus #2 + minimum #2)

"CHF 2'500.-" → 2500.0 "3.5 Zimmer" → 3.5 "85 m²" → 85.0

Compiled patterns reused across rows · Swiss thousands separator handled · None on parse failure

SQLite database with reusable SQL queries

bonus #3 — database + SQL

Schema

canton_stats

canton (PK), name, mean_rent_4room,
mean_rent_per_m2, population

listings

listing_id (PK), canton (FK),
rooms, living_space_m2, rent_chf,
price_per_m2, is_urban, lat, lng

Example query

```
SELECT l.canton, cs.canton_name,  
COUNT(*) AS n_listings,  
ROUND(AVG(l.rent_chf), 0)  
    AS avg_rent_chf,  
ROUND(AVG(l.price_per_m2), 2)  
    AS avg_price_per_m2  
FROM listings AS l  
LEFT JOIN canton_stats AS cs  
ON cs.canton = l.canton  
GROUP BY l.canton, cs.canton_name  
ORDER BY avg_price_per_m2 DESC;
```

Three statistical tests — all reject H_0

All p -values $\ll 0.05$ at $\alpha = 0.05$ significance

PEARSON CORRELATION

Rent vs. living space

$$r = 0.799$$

$$p = 4.0 \times 10^{-145}$$

Strong positive linear relationship

WELCH T-TEST

Price/m² urban vs. rural cantons

$$t = 11.55$$

$$p = 4.7 \times 10^{-23}$$

Cohen's $d = 1.21$ — large urban premium

ONE-WAY ANOVA

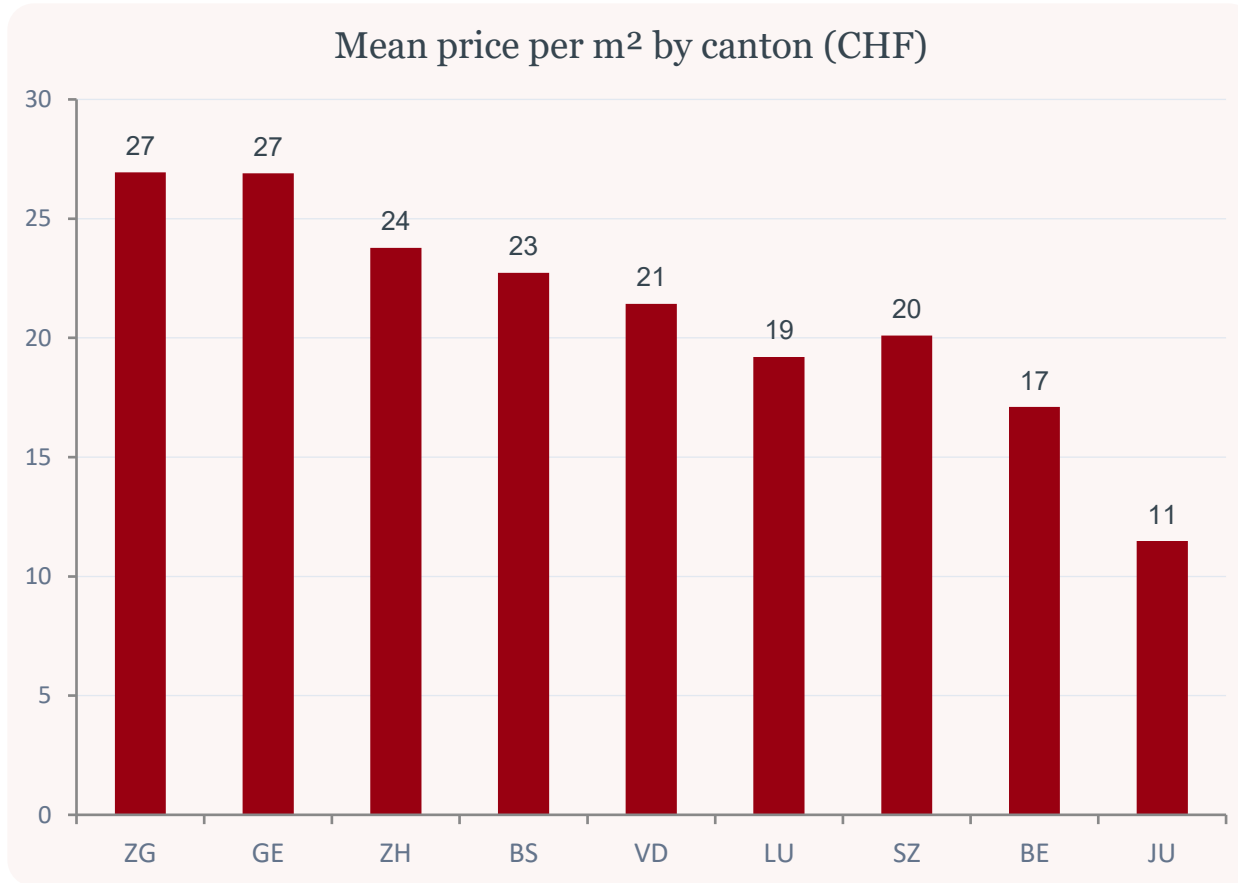
Price/m² across 26 cantons

$$F = 40.10$$

$$p = 2.6 \times 10^{-112}$$

$\eta^2 = 0.62$ — canton membership matters

Visualising the Swiss rental landscape



folium

interactive map: 650+ markers, colour-graded by price/m²

choropleth

canton-level heat map of mean rent

matplotlib + seaborn

scatter, boxplot, violin, histogram

Streamlit

the same charts, interactive in the browser

Streamlit web app + LLM market commentary

bonus #4 (LLM) + bonus #5 (web app)

Streamlit dashboard

Sidebar filters

canton · rooms · price · data source

Headline metrics

4 KPIs computed live from filtered data

Tabs for each angle

overview · stats · map · LLM

Live SQL

every chart re-runs the SQLite query

Embedded folium map

HTML rendered inside the dashboard

LLM commentary

Together.ai · Mistral 7B Instruct

Aggregated tables (not raw rows) are sent to the LLM as a structured prompt with explicit instructions: highlight the most/least expensive cantons, the room–rent relation, the urban–rural gap, plus one renter–takeaway sentence.

If TOGETHER_API_KEY is missing, the helper returns a deterministic offline summary so the demo always runs.

Conclusions

Size is king

Pearson $r = 0.80$ — living space alone explains most of the rent variance

Cantons matter

ANOVA reveals strong cross-canton differences ($\eta^2 = 0.62$)

Urban premium

Welch t-test: urban CHF 22.3/m² vs. rural CHF 16.3/m² ($d = 1.21$)

Listings $\approx$ BFS

Live asking prices broadly align with the official mean — but with non-trivial dispersion

Limitations: BFS reference is annual and lagging · Homegate snapshot biases toward what is currently available · canton-level analysis hides commune-level variation.

Appendix — minimum requirements (8 pts)

#	Requirement	Where in the project
1	Real-world data	app/api_client.py · app/scrapper.py
2	Data preparation (regex)	app/data_cleaning.py
3	Built-in structures + pandas	Throughout — notebooks 1–3
4	Conditionals + loops + break/continue	scraper.py, data_cleaning.py
5	Procedural and OOP	Functional pipeline + app/models.py
6	Tables + visualisations	app/visualization.py · notebook 3
7	Statistical analysis with p-value	app/analysis.py — 3 tests
8	Code on Moodle / GitHub	github.com/Marko-Vukcevic/swiss-rent-radar

Appendix — bonus points (6 pts)

#	Bonus	Where in the project
1	Creativity	BFS + scrape combo · choropleth · LLM commentary
2	Web scraper / API	scraper.py (BeautifulSoup) + api_client.py
3	Database + SQL	app/database.py · 4 reusable queries
4	LLM	app/llm_helper.py — Together.ai Mistral 7B
5	Web app	app/streamlit_app.py
6	Public GitHub	github.com/Marko-Vukcevic/swiss-rent-radar

Thank you · Questions?

github.com/Marko-Vukcevic/swiss-rent-radar · Kermo Aladin & Vukcevic Marko